

email_sender.py

Python

```
def send_email(recipient, subject, body):  
    return f"Email sent to {recipient} with subject: '{subject[:20]}...'"
```

Importing from Packages

To use modules and functions within a package, you use the dot (.) notation to specify the package and module hierarchy.

Example: main.py

Python

--- File: main.py ---

```
# 1. Importing a specific function from a module inside the package  
from reporting.report_generator import generate_summary_report
```

```
# 2. Importing the entire module from the package  
import reporting.email_sender
```

```
data = [1, 2, 3, 4, 5, 6]
```

```
# Using the function imported directly  
summary = generate_summary_report(data)  
print(f"Report Output: {summary}")
```

```
# Using the function imported via the module path  
email_result = reporting.email_sender.send_email(  
    recipient="admin@corp.com",  
    subject="Weekly Sales Report",  
    body="Attached is the detailed report..."  
)  
print(f"Email Status: {email_result}")
```

7. Object-Oriented Programming (OOP) - The Core Pillars

7.1 Classes and Objects

Defining a Class (The Blueprint)

A **Class** is a blueprint or a template for creating objects. It defines a set of attributes (data/variables) and methods (functions) that the created objects will possess.

Real-time Example: A Vehicle class.

Creating an Object (The Instance)

An **Object** (or instance) is a concrete entity built from the class blueprint. Each object has its own unique set of attribute values.

Python

1. Defining the Class

class Vehicle:

Class Variable (shared by all instances)
 WHEELS = 4

2. Creating an Object (Instance) - This calls the __init__ method

def __init__(self, make, model, year):
 # Instance Variables (unique to each instance)
 self.make = make
 self.model = model
 self.year = year
 self.is_running = False # Initial state

Instance Method
def start_engine(self):
 self.is_running = True
 return f'{self.make} {self.model}'s engine started."

def get_details(self):
 return f'{self.year} {self.make} {self.model} (Wheels: {Vehicle.WHEELS})"

Creating Objects (Instances)

car1 = Vehicle("Toyota", "Camry", 2022)
car2 = Vehicle("Honda", "Civic", 2024)

Accessing methods and attributes

print(car1.get_details())
print(car2.start_engine())

Instance Variables vs. Class Variables

Variable Type	Definition	Access/Scope	Real-time Use Case
Instance Variable	Defined inside __init__ using self.variable = value.	Unique to each object/instance.	car1.model, car2.year
Class Variable	Defined outside __init__ at the class level.	Shared among all objects of the class. Accessed via ClassName.variable.	Vehicle.WHEELS (All cars have 4 wheels).

Python

Accessing

print(f'Car 1 Model: {car1.model}') # Instance Variable
print(f'Car 2 Wheels: {car2.WHEELS}') # Accessing Class Variable via instance (good)

Modifying the Class Variable (affects all instances)

Vehicle.WHEELS = 6
print(f'After modification, Car 1 Wheels: {car1.WHEELS}') # 6

The self Variable

The `self` variable is a reference to the **instance** of the class currently being operated on. It is the first argument in all instance methods (including `__init__`). It is how an instance method accesses the instance's attributes.

When you call `car1.start_engine()`, Python automatically passes `car1` as the `self` argument to the method.

Constructors (`__init__`) and Destructors (`__del__`)

- **`__init__(self, ...)` (Constructor):** This method is automatically called when a new object is created. Its main purpose is to **initialize** the instance's attributes.
- **`__del__(self)` (Destructor):** This method is called when an object's reference count drops to zero and it is about to be garbage-collected. It is rarely used in modern Python, but can be useful for **cleanup** operations (e.g., closing file handles, database connections).

Python

```
class DatabaseConnection:
```

```
    def __init__(self, host):
        self.host = host
        print(f"Connection established to {self.host}.") # Resource allocation
```

```
    def __del__(self):
        # Resource deallocation/cleanup
        print(f"Connection to {self.host} closed and object destroyed.")
```

```
# Constructor called
```

```
db_conn = DatabaseConnection("production_server")
```

```
print("Working with the database...")
```

```
# Destructor is called automatically when the object is no longer referenced
```

```
del db_conn
```

```
# Or when the program exits
```

7.2 Encapsulation

Encapsulation is the principle of bundling data (attributes) and the methods (functions) that operate on that data into a single unit (the class). It also involves **data hiding**, restricting direct access to an object's internal state.

Bundling Data and Methods Together

We achieved this in the `Vehicle` class by having attributes (`make`, `model`) and methods (`start_engine`, `get_details`) together.

Access Specifiers (Public, Protected, Private Conventions)

Python does not have strict access specifiers like C++ or Java. Instead, it uses **naming conventions** to suggest access intent:

1. **Public (No prefix):** Accessible from anywhere. (e.g., `self.make`)
2. **Protected (Single underscore `_`):** Intended for internal use, especially within the class and its subclasses. Should be treated as internal by convention. (e.g., `self._engine_id`)
3. **Private (Double underscore `__`):** Triggers **name mangling** (Python changes the name to `_ClassName__variable`) to make it harder to access from outside the class, although technically still possible. This is used for attributes you *really* don't want external code touching. (e.g., `self.__password`)

Python

```
class BankAccount:
```

```
    def __init__(self, balance):
        # Public: Accessible directly
        self.account_type = "Savings"
        # Protected: Conventionally internal
        self._minimum_balance = 100
        # Private: Name-mangled (stronger hiding)
        self.__balance = balance # Store sensitive data privately
```

```
    def get_balance(self): # Public accessor method
        return f"Current balance: ${self.__balance:.2f}"
```

```
# Object creation
```

```
user_acc = BankAccount(1000.00)
```

```
# Access
```

```
print(f"Type: {user_acc.account_type}") # Accessing Public variable (OK)
```

```
# Attempt to access protected/private variables directly
```

```
print(f"Minimum (Protected): {user_acc._minimum_balance}") # Access possible, but bad practice
```

```
# print(f"Balance (Private): {user_acc.__balance}") # ERROR: AttributeError
```

```
# Accessing private attribute via name mangling (Discouraged!)
```

```
print(f"Balance (Mangled Access): {user_acc._BankAccount__balance}") # Accesses the mangled name
```

Using Getters and Setters (Property Decorators)

To control access to internal attributes (like `__balance`), we use public methods called **Getters** (to read the data) and **Setters** (to modify the data, often with validation logic).

Python's **@property decorator** allows you to define methods that can be accessed like attributes, making the code cleaner while still enforcing control.

Python

```
class BankAccount:
```

```
    def __init__(self, balance):
        self.__balance = balance # Private
```

```
    # Getter: Allows reading the balance as an attribute (user_acc.balance)
```

```
    @property
```

```
    def balance(self):
        return self.__balance
```

```
    # Setter: Allows setting the balance as an attribute (user_acc.balance = 500)
```

```
    @balance.setter
```

```
    def balance(self, new_amount):
        if new_amount < 0:
            raise ValueError("Balance cannot be negative.")
        self.__balance = new_amount
        print("Balance updated successfully.")
```

```
user_acc = BankAccount(1000)
```

```
# 1. Using the getter method (accessed like an attribute)
```

```
print(f"Initial Balance: {user_acc.balance}")
```

```
# 2. Using the setter method (accessed like an attribute with validation)
user_acc.balance = 500
# user_acc.balance = -100 # This would raise the ValueError
```

7.3 Inheritance

Inheritance is a mechanism that allows a new class (the **subclass** or **derived class**) to inherit properties (attributes and methods) from an existing class (the **superclass** or **base class**). This promotes code reuse.

Single, Multi-Level, and Multiple Inheritance (MRO)

1. **Single Inheritance:** A subclass inherits from only one superclass.
2. **Multi-Level Inheritance:** A class inherits from a subclass, which in turn inherited from another class (e.g., $A \rightarrow B \rightarrow C$).
3. **Multiple Inheritance:** A subclass inherits from two or more superclasses. This can lead to the **Diamond Problem** if methods are duplicated.

Python uses the **Method Resolution Order (MRO)** (specifically the C3 Linearization algorithm) to determine the order in which base classes are searched for methods. You can view the MRO using `ClassName.mro()`.

Python

```
# Single Inheritance Example
class Employee: # Base Class
    def __init__(self, name):
        self.name = name

    def get_info(self):
        return f"Employee: {self.name}"

class Developer(Employee): # Subclass
    def __init__(self, name, language):
        super().__init__(name) # Call parent constructor
        self.language = language

    def code(self):
        return f"{self.name} is coding in {self.language}."

dev1 = Developer("Alice", "Python")
print(dev1.get_info()) # Inherited from Employee
print(dev1.code())    # Specific to Developer
```

Method Overriding

Method overriding occurs when a subclass provides a specific implementation for a method that is already defined in its superclass. This allows the subclass to tailor inherited behavior.

Python

```
class Manager(Employee):
    def __init__(self, name, team_size):
        super().__init__(name)
        self.team_size = team_size

# Overriding the get_info method from the Employee class
def get_info(self):
```

```
# You can call the superclass method using super()
parent_info = super().get_info()
return f"{parent_info} (Manager of {self.team_size} people)."
```

```
mgr1 = Manager("Bob", 5)
print(mgr1.get_info()) # Calls the overridden Manager method
```

Using the super() function

The super() function is used to give access to methods and attributes of the parent or sibling class. It is most commonly used in __init__ to call the parent's constructor and initialize inherited attributes.

7.4 Polymorphism

Polymorphism means "many forms." In OOP, it refers to the ability of different objects to respond to the same method call in their own way.

Method Overloading (Simulated)

Method Overloading means defining multiple methods in the same class with the same name but different numbers or types of arguments.

Python does **not** support traditional method overloading. However, it can be simulated using **default arguments** or variable-length arguments (*args, **kwargs).

Python

```
class Calculator:
    # Simulating overloading with default arguments
    def add(self, a, b, c=None):
        if c is None:
            # Handles two arguments
            return a + b
        else:
            # Handles three arguments
            return a + b + c
```

```
calc = Calculator()
print(f"2 arguments: {calc.add(5, 10)}") # 15
print(f"3 arguments: {calc.add(5, 10, 15)}") # 30
```

Method Overriding (Run-time Polymorphism)

As demonstrated in the Inheritance section, method overriding allows a derived class to replace a method's implementation. When the method is called on an object, the decision of which version to execute is made at **run-time** based on the object's actual type.

Python

```
class Animal:
    def speak(self):
        raise NotImplementedError("Subclass must implement abstract method")

class Dog(Animal):
    def speak(self):
        return "Woof!"
```

```

class Cat(Animal):
    def speak(self):
        return "Meow!"

# Polymorphic list: objects of different classes
animals = [Dog(), Cat()]

# The same method call (speak()) yields different results
for animal in animals:
    print(f"The animal says: {animal.speak()}")

```

Operator Overloading (e.g., using `__add__` to redefine `+`)

Python allows you to redefine how standard operators (like `+`, `-`, `*`) behave when applied to instances of your custom class. This is done by implementing special methods (Dunder methods, like `__add__`).

```

Python
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

# Operator Overloading for the '+' operator
def __add__(self, other):
    # Defines the operation V1 + V2
    new_x = self.x + other.x
    new_y = self.y + other.y
    return Vector(new_x, new_y)

def __repr__(self):
    return f"Vector({self.x}, {self.y})"

v1 = Vector(5, 10)
v2 = Vector(3, 2)

# Using the '+' operator calls the __add__ method
v3 = v1 + v2
print(f"Vector V3: {v3}") # Output: Vector(8, 12)

```

7.5 Data Abstraction

Data Abstraction means showing only essential information to the user and hiding the complex implementation details.

- The user of the `BankAccount` class only interacts with `user_acc.balance` and does not need to know the attribute is internally stored as `self.__balance` or how the validation logic in the setter works.

Using the `abc` module (Abstract Base Classes)

Python achieves formal abstraction using the built-in `abc` (Abstract Base Classes) module. This allows you to define a blueprint class that **must** have certain methods implemented by any concrete subclass.

- An **Abstract Class** cannot be instantiated directly.
- An **Abstract Method** is a method declared in the abstract class but has no implementation (no body).

Python

```
from abc import ABC, abstractmethod
```

```
# Abstract Class: Cannot be instantiated
```

```
class PaymentProcessor(ABC):
```

```
    # Concrete Method (has implementation)
```

```
    def process_transaction(self, amount):
```

```
        print(f"--- Processing ${amount} transaction ---")
```

```
        return self._authorize_payment(amount) # Calls the abstract method
```

```
    # Abstract Method: Must be implemented by subclasses
```

```
    @abstractmethod
```

```
    def _authorize_payment(self, amount):
```

```
        pass # No implementation here
```

```
# Concrete Subclass: Must implement the abstract method
```

```
class CreditCardProcessor(PaymentProcessor):
```

```
    def _authorize_payment(self, amount):
```

```
        # Real-world: Connects to a payment gateway API
```

```
        print("Authorizing via credit card gateway...")
```

```
        if amount < 500:
```

```
            return "SUCCESS: CC authorized"
```

```
        return "FAILURE: CC limit exceeded"
```

```
# Example Usage
```

```
# processor = PaymentProcessor() # TypeError: Can't instantiate abstract class
```

```
cc_proc = CreditCardProcessor()
```

```
status = cc_proc.process_transaction(450.00)
```

```
print(f"Transaction Status: {status}")
```

```
# If CreditCardProcessor failed to implement _authorize_payment,
```

```
# Python would raise a TypeError when trying to instantiate it.
```

8. Errors, Exceptions, and File Handling

8.1 Errors and Exceptions

Syntax Errors (Compile-time)

A **Syntax Error** occurs when the code violates the grammatical rules of the Python language. The interpreter cannot parse the code and will stop execution before the program even starts.

Real-time Example: Missing a colon, parenthesis, or having improper indentation.

Python

```
# --- Syntax Error Example ---
```

```
# if 5 > 2    <-- SyntaxError: expected ':'
```

```
# print("Five is greater than two")
```

```
# print("Hello" <-- SyntaxError: unclosed parenthesis
```

Logical Errors and Runtime Errors (Exceptions)